Established – 1961 Subject:OSDBMS

SEVA SADAN'S

R. K. TALREJA COLLEGE

OF

ARTS, SCIENCE & COMMERCE ULHASNAGAR – 421003

CERTIFICATE

This is to certify that Mr./Ms. DEVENDRA SUNIL PATIL of S.Y. Information Technology (SYIT) Roll No. 2542033 has satisfactorily completed the Open Source DataBase Management System Mini Project Product Review and Rating Management entitled

during the academic year 2025 – 2026, as a part of the practical requirement. The project work is found to be satisfactory and is approved for submission.

PROF. INCHARGE	HEAD OF DEPT
Sahil Shukla	**Laxmi Jeswani**
_____	_____

INDEX

Sr. No.	Chapter Title	PAGE NO
1	Introduction	3-4
2	Problem Definition	5-6
3	Objectives of the Project	7-9
4	Scope of the Project	10-11
5	Requirement Specification	12
6	System Design	13-16
7	Database Design	17
8	UML Diagrams	18-21
9	SQL Implementation	22-30
10	System Testing and Result	31-33
11	Security, Backup and Recovery	34-36
12	Future Scope and Conclusion	37
13	References	38
14	Glossary	39

PROJECT REPORT

Product Review and Rating Management System

CHAPTER 1: INTRODUCTION

The **Product Review and Rating Management System** is a comprehensive, database-driven application designed to efficiently store, manage, and analyze product reviews and ratings in a structured, secure, and reliable manner. In today's highly competitive digital marketplace, customer feedback plays a vital role in shaping purchasing decisions, building brand trust, and improving product quality. Online reviews and star ratings significantly influence consumer behavior, making it essential for businesses to maintain an organized and scalable system for handling such data.

This system enables registered users to submit detailed reviews and numerical ratings for various products. Each review typically includes user information, product details, rating scores, review text, timestamps, and optional metadata. The ratings are dynamically aggregated using SQL aggregate functions such as AVG(), COUNT(), and SUM() to compute real-time average product ratings, total number of reviews, and overall rating distributions. These calculations help provide meaningful insights into product performance and customer satisfaction.

The entire system is implemented using **MySQL**, an open-source relational database management system known for its robustness, performance optimization, scalability, and security features. MySQL provides a structured environment where data is organized into relational tables connected through primary and foreign keys. This relational design ensures data integrity and eliminates redundancy.

A major focus of this project is the application of core database management concepts. The database schema is carefully designed and normalized up to **Third Normal Form (3NF)** to reduce data redundancy and dependency issues. The design typically includes separate tables for users, products, reviews, and ratings, each with clearly defined relationships. This normalization ensures efficient storage, easy maintenance, and improved query performance.

Transaction management is another critical component of the system. By implementing MySQL transactions using commands such as BEGIN, COMMIT, and ROLLBACK, the system ensures atomicity, consistency, isolation, and durability (ACID properties). If an error occurs during review submission—such as invalid data entry or system interruption—the transaction is rolled back to prevent partial or inconsistent data from being stored. This guarantees database consistency and reliability.

Concurrency control mechanisms provided by MySQL allow multiple users to submit reviews simultaneously without causing data conflicts or inconsistencies. Through row-level locking and isolation levels, the system prevents issues such as dirty reads, lost updates, and phantom reads. This ensures accurate aggregation of ratings even when the system experiences high traffic.

In addition to basic functionality, the system can incorporate advanced features such as:

- Role-based access control (RBAC) to differentiate permissions between users and administrators.
- Review moderation mechanisms to filter inappropriate or spam content.
- Search and filtering capabilities based on rating, date, or product category.
- Indexing strategies to improve query performance for large datasets.
- Backup and recovery strategies to safeguard data against loss.

The system architecture is scalable and can be integrated with web or mobile front-end applications using APIs. It can also support analytical reporting by generating insights such as top-rated products, most active users, rating trends over time, and customer sentiment patterns.

Overall, the Product Review and Rating Management System provides practical exposure to real-world database operations and demonstrates how structured data management enhances accuracy, integrity, efficiency, and decision-making. By combining relational database design principles, normalization techniques, transaction control, and concurrency management, the project showcases the importance of database systems in modern application development and digital commerce environments.

CHAPTER 2: PROBLEM DEFINITION

Managing product reviews and ratings without a structured database system creates numerous operational and technical challenges. In the absence of a well-designed relational database, data is often stored in spreadsheets, flat files, or other unorganized formats. Such approaches lead to **data redundancy**, where the same information is stored multiple times, increasing storage requirements and the likelihood of inconsistencies. For example, product details may be repeatedly entered with every review, resulting in duplication and maintenance difficulties.

Another major issue is **data inconsistency**. When information is not centralized or properly linked through relational constraints, updates made in one location may not reflect elsewhere. This can cause discrepancies in product ratings, inaccurate review counts, or outdated product information. As the volume of reviews grows, retrieving meaningful insights becomes increasingly complex and time-consuming. Manual record-keeping systems are not only inefficient but also highly prone to human error, reducing reliability and accuracy.

The lack of proper **transaction control** further complicates the system. In a real-world application, multiple operations may occur during review submission—such as validating user credentials, inserting review data, updating product rating averages, and logging timestamps. If a failure occurs during any of these steps (for example, due to an invalid rating value, server interruption, or system crash), partial updates may be saved. This results in inconsistent database states, such as a review being recorded without updating the product's average rating. Without transaction management mechanisms like COMMIT and ROLLBACK, there is no way to revert incomplete operations, potentially corrupting the integrity of the system.

In a **multi-user environment**, concurrency issues also arise. When multiple users attempt to submit reviews simultaneously, data conflicts may occur if proper concurrency control mechanisms are not implemented. Problems such as lost updates, dirty reads, or inconsistent aggregate calculations can significantly affect the accuracy of product ratings. Without row-level locking or appropriate isolation levels, the database may produce unreliable results under heavy traffic conditions.

Security is another critical concern. User information such as usernames, email addresses, and passwords must be protected against unauthorized access. Without proper authentication, authorization, and encryption mechanisms, sensitive data can be exposed to security threats. Additionally, maintaining data integrity through constraints such as primary keys, foreign keys, unique constraints, and validation rules is essential to prevent invalid or duplicate entries.

Scalability also becomes a challenge when the system grows. As the number of products and reviews increases, unstructured systems struggle with slow retrieval times and inefficient searching capabilities. There is a need for indexing, optimized queries, and structured schema design to ensure fast and reliable performance.

Therefore, a reliable database-driven system is essential for efficiently managing product reviews and ratings. By implementing structured relational design, normalization techniques, transaction handling, rollback mechanisms, concurrency control, and robust security measures, the system ensures data accuracy, consistency, integrity, and performance. Such an approach not only improves operational efficiency but also enhances user trust and supports informed decision-making in modern digital marketplaces.

CHAPTER 3: OBJECTIVES OF THE PROJECT

The **main objectives** of the Product Review and Rating Management System are designed to ensure efficient data management, reliability, and practical exposure to real-world database concepts. The objectives are expanded and explained below:

---

1. To Design and Implement a Well-Structured Relational Database Using MySQL

The primary objective of this project is to design a robust and logically structured relational database using MySQL. The database schema is carefully planned to define tables, attributes, relationships, and constraints clearly. Entity-Relationship (ER) modeling techniques are used to identify entities such as Users, Products, Categories, Reviews, and Admins, along with their relationships. The design ensures normalization up to Third Normal Form (3NF) to eliminate redundancy and dependency issues, resulting in efficient storage and maintainability.

---

2. To Store User, Product, Category, Admin, and Review Data Efficiently

Another key objective is to organize and store data systematically across multiple related tables. Each table serves a specific purpose:

- **Users table** stores user credentials and profile information.
- **Products table** stores product details such as name, description, and price.
- **Categories table** organizes products into logical groups.
- **Reviews table** stores ratings and textual feedback linked to both users and products.
- **Admin table** manages administrative access and control.

Efficient data storage ensures quick retrieval, easy updates, and better scalability as the system grows.

---

3. To Maintain Data Integrity Using Constraints

The system emphasizes strong data integrity by implementing various SQL constraints:

- **PRIMARY KEY** ensures each record is uniquely identifiable.
- **FOREIGN KEY** maintains referential integrity between related tables.
- **UNIQUE** prevents duplicate entries such as duplicate emails.
- **CHECK** constraints validate input values (e.g., rating must be between 1 and 5).

These constraints ensure accuracy, consistency, and reliability of stored data while preventing invalid or inconsistent records.

---

4. To Dynamically Calculate Average Product Ratings Using SQL Aggregate Functions

The system uses SQL aggregate functions such as AVG(), COUNT(), and SUM() to dynamically compute:

- Average rating of each product
- Total number of reviews
- Distribution of rating values

This enables real-time updates of product ratings whenever new reviews are added. Dynamic calculation ensures that users always see accurate and up-to-date rating information.

---

5. To Implement Transaction Control Using COMMIT and ROLLBACK

Transaction management is implemented to ensure database consistency and reliability. When a user submits a review, multiple operations may occur simultaneously (e.g., inserting review data and updating rating statistics). By using:

- **COMMIT** to permanently save successful transactions
- **ROLLBACK** to undo incomplete or failed operations

the system prevents partial updates and maintains data integrity. This demonstrates the practical application of ACID properties (Atomicity, Consistency, Isolation, Durability).

---

6. To Understand Concurrency Control in a Multi-User Environment

Since multiple users can submit reviews at the same time, the system is designed to handle concurrent access efficiently. By leveraging MySQL's concurrency control mechanisms such as row-level locking and isolation levels, the system prevents issues like lost updates, dirty reads, and inconsistent calculations. This objective helps in understanding how real-world applications maintain accuracy under heavy user activity.

---

7. To Gain Practical Knowledge of SQL and Real-World Database Management Concepts

The final objective of the project is educational and practical. It provides hands-on experience with:

- Writing SQL queries (SELECT, INSERT, UPDATE, DELETE)
- Implementing joins to combine data from multiple tables
- Using aggregate and group-by functions
- Applying constraints and indexes
- Managing transactions and concurrency

Through this project, students gain a deeper understanding of database design principles and real-world data management practices used in modern applications.

Overall, these objectives collectively ensure that the Product Review and Rating Management System is not only functionally effective but also academically valuable, demonstrating the importance of structured database design, integrity enforcement, transaction handling, and performance optimization in modern information systems.

CHAPTER 4: SCOPE OF THE PROJECT

The scope of the Product Review and Rating Management System is structured into two implementation levels to demonstrate both foundational database functionality and advanced database management capabilities using MySQL.

1. Database Design and Normalization

- Creation of ER diagram and relational schema
- Implementation of tables: Users, Products, Categories, Reviews, Admin
- Normalization up to Third Normal Form (3NF)
- Definition of relationships using PRIMARY and FOREIGN KEYS

2. User Registration and Authentication

- Secure storage of user credentials
- Enforcement of UNIQUE constraints (email, username)
- Validation checks for input fields
- Basic login verification through SQL queries

3. Product and Category Management

- Categorization of products using relational mapping
- Maintenance of referential integrity
- CRUD operations (Create, Read, Update, Delete) on products

4. Review and Rating Submission

- Users can submit ratings (1–5 scale) and text reviews
- Validation using CHECK constraints
- Prevention of duplicate reviews per user per product (if required)

5. Rating Aggregation

- Use of SQL aggregate functions (AVG(), COUNT(), GROUP BY)
- Dynamic calculation of average ratings
- Retrieval of review statistics

This stage ensures a fully functional backend database system suitable for academic and small-scale implementation.

CHAPTER 5: REQUIREMENT SPECIFICATION

Hardware Requirements:

• Computer/Laptop

• Minimum 4GB RAM

• 20GB free storage

Software Requirements:

• MySQL Server

• MySQL Workbench

• Windows/Linux OS

• SQL language

CHAPTER 6: SYSTEM DESIGN

The system follows a **client–server architecture**, where the client interface communicates directly with the MySQL database server to perform operations. Users interact with the system using tools such as **MySQL Workbench**, command-line interfaces (CLI), or integrated development environments (IDEs). These tools act as clients that send structured SQL commands to the MySQL server for execution.

SQL statements such as **INSERT, SELECT, UPDATE, and DELETE** are processed by the MySQL query engine. The server validates the syntax, checks user permissions, executes the query, and returns the results to the client. This separation between client and server ensures better security, scalability, and centralized data management.

Client Layer Responsibilities

The client layer is responsible for:

- Accepting user input (e.g., adding a product review or rating).
- Sending SQL queries to the MySQL server.
- Displaying query results in a readable format.
- Handling user-level validation before submitting data.

By using MySQL Workbench or CLI tools, users can:

- Execute ad-hoc queries.
- Run stored procedures.
- Monitor database performance.
- Manage database schema and data.

Server Layer Responsibilities

The MySQL server performs the core database functions, including:

- **Query Parsing and Optimization**

    - Analyzing SQL statements.
    - Creating efficient execution plans.
- **Transaction Management**
    - Ensuring operations follow ACID properties:
        - **Atomicity** – All operations in a transaction succeed or fail together.
        - **Consistency** – Database remains valid before and after transactions.
        - **Isolation** – Transactions do not interfere improperly with each other.
        - **Durability** – Committed changes are permanently stored.
- **Concurrency Control**
    - Using **row-level locking** to allow multiple users to interact with different records simultaneously.
    - Preventing conflicts when users submit reviews or update ratings at the same time.

Transaction Handling and Rollback Mechanism

Transaction handling ensures that database operations are executed completely or not at all.

For example:

- When a user submits a review and rating:
    1. The review is inserted into the *Reviews* table.
    2. The product's average rating is updated.
- If any step fails (e.g., constraint violation), the **ROLLBACK** command restores the database to its previous state.
- If all steps succeed, the **COMMIT** command permanently saves the changes.

This mechanism prevents:

- Partial updates

- Data inconsistencies
- Corrupted relationships between tables

Concurrency and Data Integrity

MySQL manages concurrent access using:

- **Row-level locking** (especially with the InnoDB storage engine), allowing multiple users to:
    - Submit reviews simultaneously.
    - Update different records without blocking the entire table.
- **Isolation levels** to control how transactions interact.
- **Foreign key constraints** to maintain referential integrity between products, users, and reviews.

This ensures:

- High system reliability
- Accurate rating calculations
- Consistent user experience even under multiple simultaneous connections

---

Scalability and Reliability

The client–server model allows:

- Multiple clients to connect to a centralized database.
- Easy integration with web or mobile applications.
- Efficient performance tuning at the server level.
- Centralized backup, monitoring, and recovery operations.

Overall Architectural Scope

The Product Review and Rating Management System's architecture ensures:

- Structured communication between client and server.
- Secure and controlled query execution.
- Reliable transaction handling with rollback support.
- Efficient concurrency management using row-level locking.
- High data consistency and integrity.

By combining structured client interaction with robust server-side processing, the system demonstrates practical implementation of database architecture principles using MySQL.

CHAPTER 7: DATABASE DESIGN

The database consists of five main tables:

- Users
- Categories
- Products
- Reviews
- Admins

The design follows Third Normal Form (3NF) to eliminate redundancy and ensure efficient data storage.

Table Description:

- **Users Table:** Stores user account details.
- **Categories Table:** Stores product category information.
- **Products Table:** Stores product details linked to categories.
- **Reviews Table:** Stores user reviews and ratings linked to users and products.
- **Admins Table:** Stores administrator login details.

Constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, and CHECK are used to maintain data integrity. The UNIQUE constraint on (user_id, product_id) ensures that a user can review a product only once.

CHAPTER 8: UML DIAGRAMS

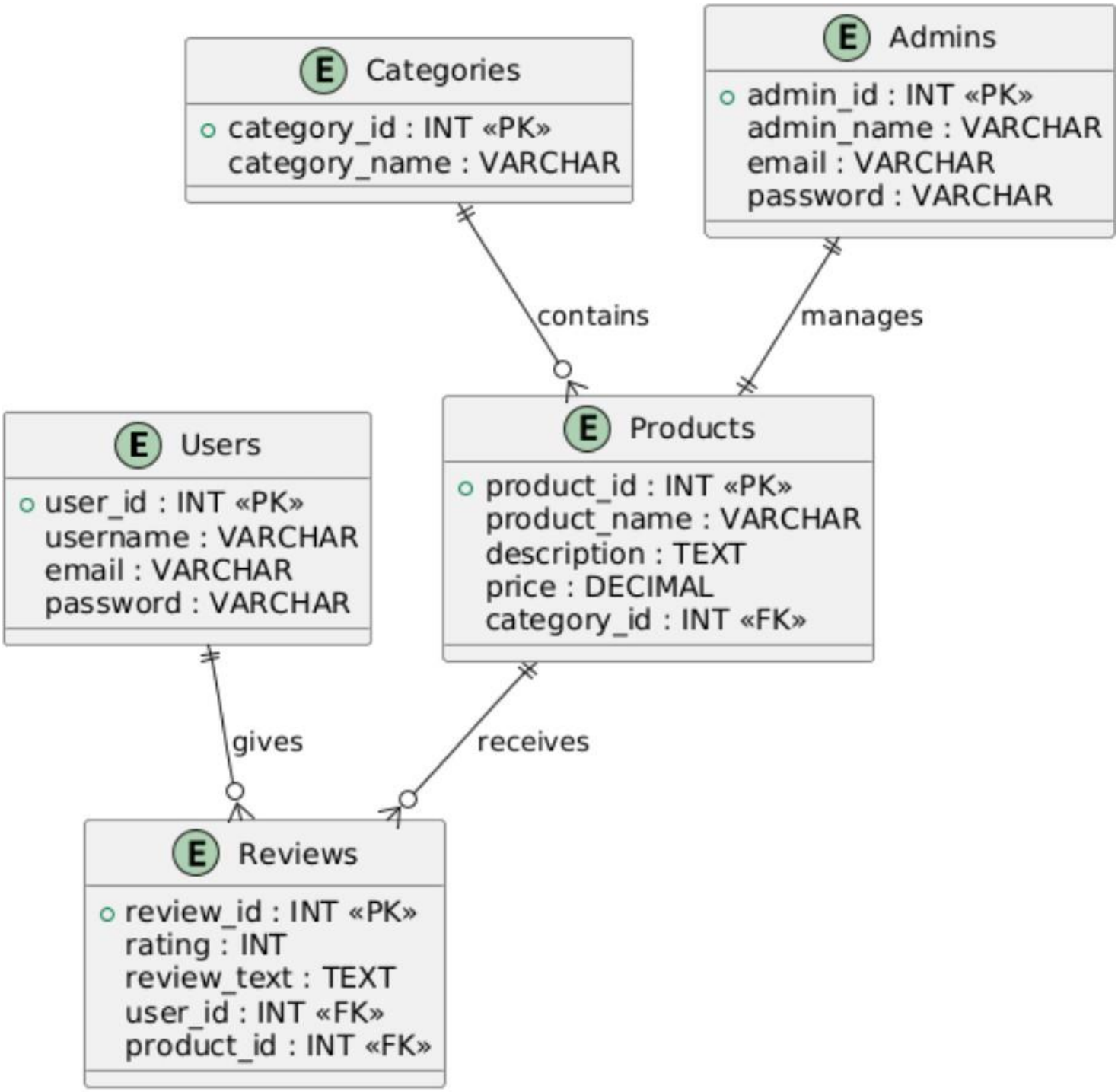

Figure 8.1 : ER [Entity Relationship] Diagram

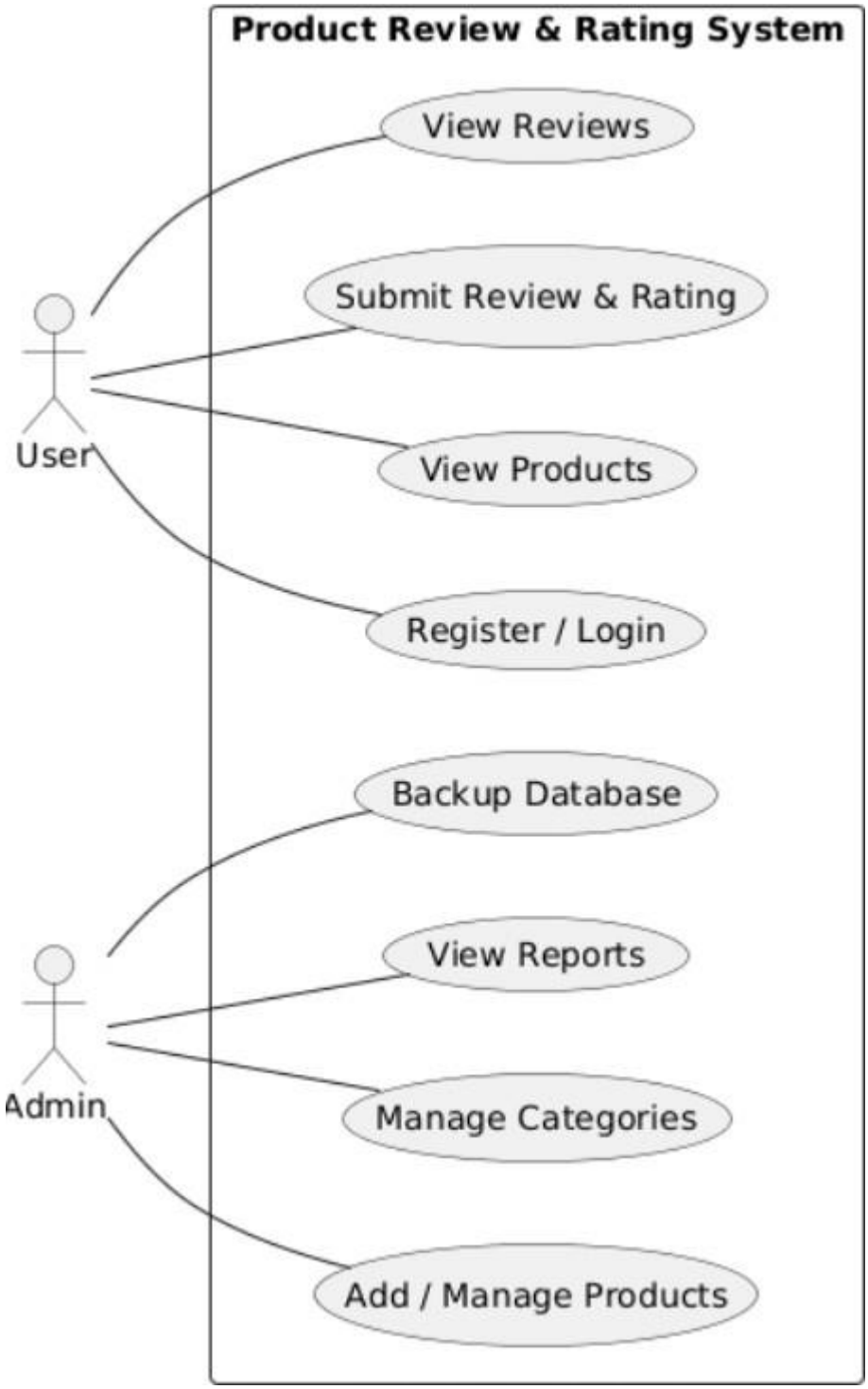

Figure 8.2 : USE CASE Diagram

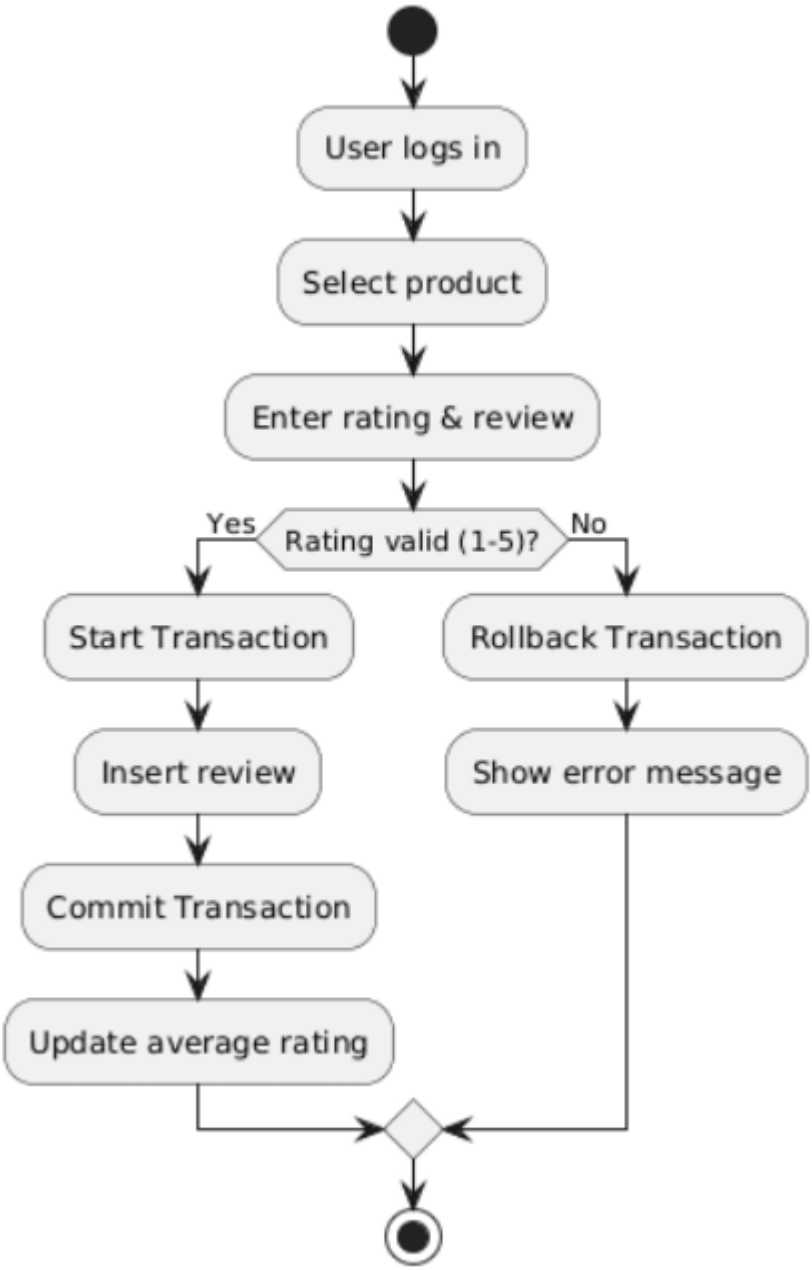

Figure 8.3 : ACTIVITY Diagram

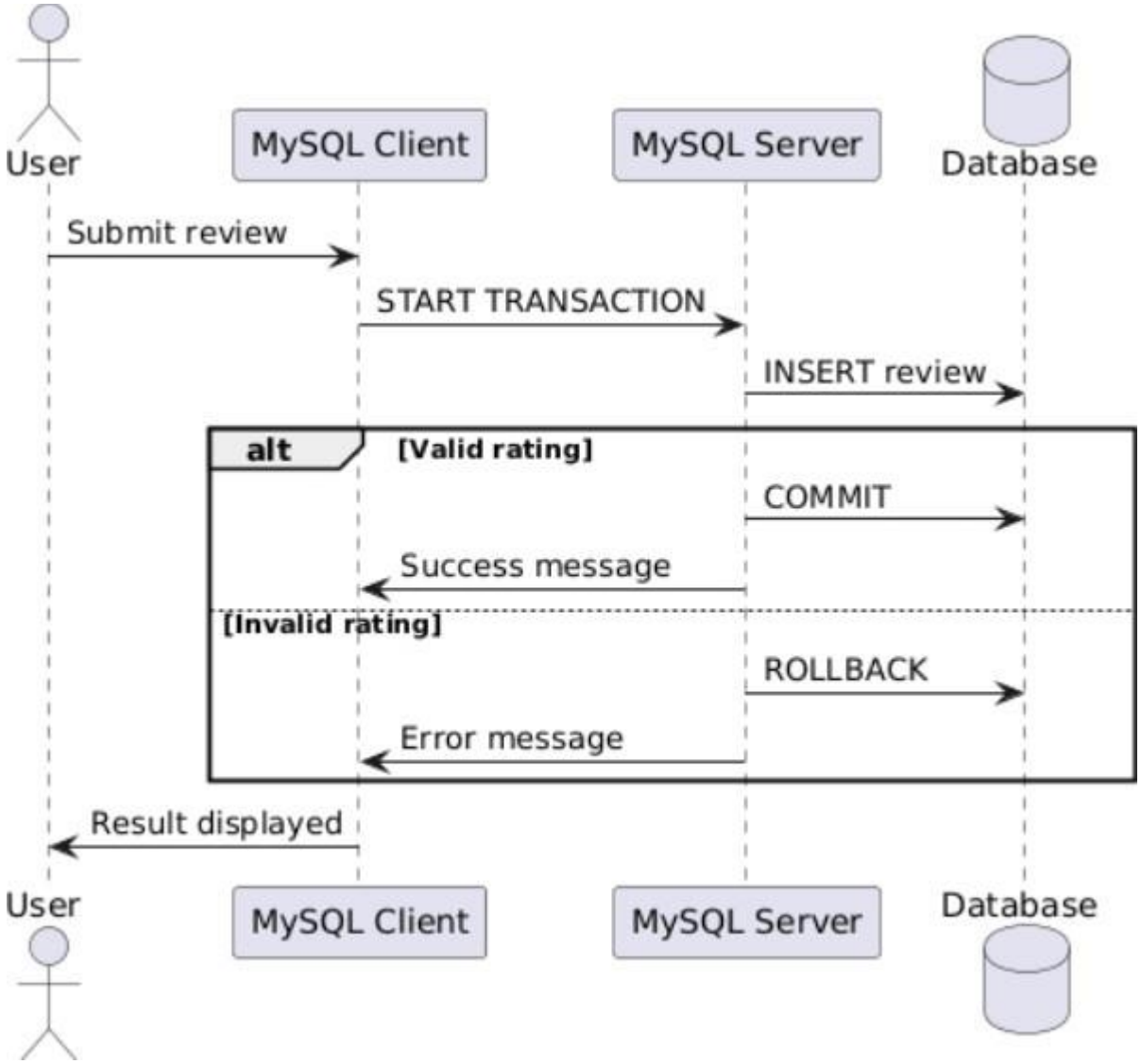

Figure 8.4 : SEQUENCE Diagram

CHAPTER 9: SQL IMPLEMENTATION

This chapter includes SQL commands for:

- Database creation
- Table creation
- Data insertion
- Transaction handling
- Data retrieval

Transactions are implemented using START TRANSACTION, COMMIT, and ROLLBACK to ensure consistency. Aggregate functions such as AVG() and COUNT() are used to dynamically calculate ratings and generate reports. JOIN queries retrieve meaningful information across multiple tables.

6.1 Create Database

Sql

```
CREATE DATABASE ProductReviewSystem;

USE ProductReviewSystem;
```

6.2 Create Tables (3NF Followed)

Users Table

Sql

```
CREATE TABLE Users (

   User_id INT PRIMARY KEY AUTO_INCREMENT,

   Username VARCHAR(50) UNIQUE NOT NULL,
```

```
    Email VARCHAR(100) UNIQUE NOT NULL,

    Password VARCHAR(100) NOT NULL,

    Created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);
```

Categories Table

Sql

```
CREATE TABLE Categories (

    Category_id INT PRIMARY KEY AUTO_INCREMENT,

    Category_name VARCHAR(100) NOT NULL

);
```

Products Table

Sql

```
CREATE TABLE Products (

    Product_id INT PRIMARY KEY AUTO_INCREMENT,

    Product_name VARCHAR(100) NOT NULL,

    Description TEXT,

    Price DECIMAL(10,2),

    Category_id INT,

    Created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (category_id) REFERENCES Categories(category_id)

);
```

Reviews Table

Sql

```
CREATE TABLE Reviews (

    Review_id INT PRIMARY KEY AUTO_INCREMENT,

    User_id INT,

    Product_id INT,

    Rating INT CHECK (rating BETWEEN 1 AND 5),

    Review_text TEXT,

    Review_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id) REFERENCES Users(user_id),

    FOREIGN KEY (product_id) REFERENCES Products(product_id),

    UNIQUE (user_id, product_id)

);
```

Admins Table

Sql

```
CREATE TABLE Admins (

    Admin_id INT PRIMARY KEY AUTO_INCREMENT,

    Admin_name VARCHAR(50),

    Email VARCHAR(100),

    Password VARCHAR(100)

);
```

6.3 Insert Sample Data (DML)

Users

Sql

```
INSERT INTO Users (username, email, password)

VALUES

('rahul', 'rahul@gmail.com', 'rahul123'),

('neha', 'neha@gmail.com', 'neha123');
```

Categories

Sql

```
INSERT INTO Categories (category_name)

VALUES

('Electronics'),

('Books'),

('Clothing');
```

Products

Sql

```
INSERT INTO Products (product_name, description, price, category_id)

VALUES

('Smartphone', '5G Android Phone', 15000, 1),
```

```
('DBMS Book', 'Database Concepts Book', 500, 2);
```

Admin

Sql

```
INSERT INTO Admins (admin_name, email, password)

VALUES ('Admin1', 'admin@gmail.com', 'admin123');
```

6.4 TRANSACTION HANDLING

Successful Transaction

Sql

```
START TRANSACTION;



INSERT INTO Reviews (user_id, product_id, rating, review_text)

VALUES (1, 1, 5, 'Excellent phone');



COMMIT;
```

Rollback Scenario (Failure Case)

Sql

```
START TRANSACTION;



INSERT INTO Reviews (user_id, product_id, rating, review_text)
```

```
VALUES (2, 1, 6, ‘Invalid rating’);  -- rating > 5 (fails)



ROLLBACK;
```

No partial update occurs

6.5 Insert Valid Reviews

Sql

```
INSERT INTO Reviews (user_id, product_id, rating, review_text)

VALUES

(2, 1, 4, 'Good but battery is average'),

(1, 2, 5, 'Very helpful book');
```

6.6 SELECT QUERIES (DQL)

View Reviews of a Product

Sql

```
SELECT u.username, r.rating, r.review_text

FROM Reviews r

JOIN Users u ON r.user_id = u.user_id

WHERE r.product_id = 1;
```

Average Rating (Dynamic Aggregation)

Sql

```
SELECT p.product_name, AVG(r.rating) AS average_rating

FROM Products p

JOIN Reviews r ON p.product_id = r.product_id

GROUP BY p.product_id;
```

Products with Avg Rating ≥ 4

Sql

```
SELECT p.product_name, AVG(r.rating) AS avg_rating

FROM Products p

JOIN Reviews r ON p.product_id = r.product_id

GROUP BY p.product_id

HAVING AVG(r.rating) >= 4;
```

Reviews by Specific User

Sql

```
SELECT p.product_name, r.rating, r.review_text

FROM Reviews r

JOIN Products p ON r.product_id = p.product_id

WHERE r.user_id = 1;
```

Top Rated Product

Sql

```
SELECT p.product_name, AVG(r.rating) AS avg_rating

FROM Products p

JOIN Reviews r ON p.product_id = r.product_id

GROUP BY p.product_id

ORDER BY avg_rating DESC

LIMIT 1;
```

Total Reviews per Product

Sql

```
SELECT product_id, COUNT(*) AS total_reviews

FROM Reviews

GROUP BY product_id;
```

6.7 UPDATE QUERY

Sql

```
UPDATE Reviews

SET rating = 5, review_text = 'Updated review'

WHERE user_id = 2 AND product_id = 1;
```

6.8 DELETE QUERY

Sql

```
DELETE FROM Reviews

WHERE review_id = 3;
```

6.9 CONCURRENCY NOTE

```
-- Multiple users submitting reviews simultaneously

-- MySQL handles concurrency using row-level locking
```

6.10 NORMALIZATION (3NF CONFIRMED)

Users → user details only

Products → product details only

Categories → category names only

Reviews → review data only

✓ No redundancy

✓ No transitive dependency

CHAPTER 10: SYSTEM TESTING AND RESULT

The system was tested using carefully prepared sample data to validate functionality, performance, and data integrity. The testing process was designed to ensure that all database operations behave as expected under normal and exceptional conditions.

Functional Testing of CRUD Operations

All core CRUD (Create, Read, Update, Delete) operations were executed successfully:

- **Create(INSERT):**
  Sample users, products, and reviews were added to the database. Foreign key constraints ensured that reviews could only be submitted for existing products by registered users.
- **Read(SELECT):**
  Queries were tested to retrieve:
  - Product details with average ratings
  - User-specific review history
  - Top-rated and recently reviewed products
- **Update(UPDATE):**
  Existing reviews and ratings were modified to verify that changes were correctly reflected in related calculations (e.g., updated average rating).
- **Delete(DELETE):**
  Reviews were removed to confirm:
  - Referential integrity was preserved
  - Dependent calculations were updated accordingly

All CRUD operations performed accurately without data corruption or unexpected behavior.

Transaction and Rollback Testing

Transaction management was tested to ensure atomicity and consistency.

Test scenarios included:

- Executing multi-step transactions (e.g., inserting a review and updating product rating).
- Intentionally introducing errors (such as invalid foreign key references or constraint violations).
- Verifying that the **ROLLBACK** command restored the database to its previous consistent state.
- Confirming that **COMMIT** permanently saved successful transactions.

The results demonstrated that:

- Partial updates were successfully prevented.
- Failed operations did not leave inconsistent data.
- The system adhered to ACID transaction properties.

Constraint Validation Testing

Various constraints were tested to ensure data accuracy and reliability:

- **NOT NULL constraints** prevented missing essential fields.
- **UNIQUE constraints** avoided duplicate entries (e.g., duplicate usernames or multiple reviews for the same product by the same user).
- **FOREIGN KEY constraints** enforced valid relationships between tables.
- **CHECK constraints (where applicable)** validated rating ranges and input rules.

Invalid data entries were rejected by the system, confirming strong validation at the database level.

Aggregation and Reporting Queries

Aggregation functions such as:

- AVG() for calculating average ratings

- COUNT() for total reviews per product
- GROUP BY for product-wise summaries

were tested using sample datasets. The outputs were verified for accuracy and consistency after insertions, updates, and deletions.

Concurrency Testing

Concurrent access scenarios were simulated by executing multiple transactions simultaneously. Testing confirmed that:

- Row-level locking prevented data conflicts.
- Simultaneous review submissions did not interfere with each other.
- Data consistency was maintained under concurrent operations.
- No deadlocks or race conditions affected system performance during controlled tests.

Overall Testing Outcome

The testing phase confirmed that:

- All CRUD operations function correctly.
- Transaction rollback mechanisms effectively prevent partial or inconsistent updates.
- Constraint enforcement blocks invalid data entries.
- Aggregation queries produce accurate results.
- The system maintains integrity and consistency during concurrent access.

These results demonstrate that the Product Review and Rating Management System is reliable, robust, and ready for practical implementation within the defined project scope.

Screenshots:

```
mysql> SELECT u.username, r.rating, r.review_text
    -> FROM Reviews r
    -> JOIN Users u ON r.user_id = u.user_id
    -> WHERE r.product_id = 1;
+----------+--------+----------------------------+
| username | rating | review_text                |
+----------+--------+----------------------------+
| neha     |      4 | Good but battery is average |
| rahul    |      5 | Excellent phone            |
+----------+--------+----------------------------+
2 rows in set (0.272 sec)
```

```
mysql> SELECT p.product_name, AVG(r.rating) AS average_rating
    -> FROM Products p
    -> JOIN Reviews r ON p.product_id = r.product_id
    -> GROUP BY p.product_id;
+--------------+----------------+
| product_name | average_rating |
+--------------+----------------+
| Smartphone   |         4.5000 |
| DBMS Book    |         5.0000 |
+--------------+----------------+
2 rows in set (0.414 sec)
```

```
mysql> SELECT p.product_name, AVG(r.rating) AS avg_ra
    -> FROM Products p
    -> JOIN Reviews r ON p.product_id = r.product_id
    -> GROUP BY p.product_id
    -> HAVING AVG(r.rating) >= 4;
+--------------+------------+
| product_name | avg_rating |
+--------------+------------+
| Smartphone   |     4.5000 |
| DBMS Book    |     5.0000 |
+--------------+------------+
2 rows in set (0.256 sec)
```

```
mysql> SELECT p.product_name, r.rating, r.review_text
    -> FROM Reviews r
    -> JOIN Products p ON r.product_id = p.product_id
    -> WHERE r.user_id = 1;
+--------------+--------+------------------+
| product_name | rating | review_text      |
+--------------+--------+------------------+
| Smartphone   |      5 | Excellent phone  |
| DBMS Book    |      5 | Very helpful book |
+--------------+--------+------------------+
2 rows in set (0.043 sec)
```

```
mysql> SELECT p.product_name, AVG(r.rating) AS avg_rating
    -> FROM Products p
    -> JOIN Reviews r ON p.product_id = r.product_id
    -> GROUP BY p.product_id
    -> ORDER BY avg_rating DESC
    -> LIMIT 1;
+--------------+------------+
| product_name | avg_rating |
+--------------+------------+
| DBMS Book    |     5.0000 |
+--------------+------------+
1 row in set (0.078 sec)
```

```
mysql> SELECT product_id, COUNT(*) AS total_reviews
    -> FROM Reviews
    -> GROUP BY product_id;
+------------+---------------+
| product_id | total_reviews |
+------------+---------------+
|          1 |             2 |
|          2 |             1 |
+------------+---------------+
2 rows in set (0.180 sec)
```

CHAPTER 11: SECURITY, BACKUP AND RECOVERY

Security and data protection are critical components of the Product Review and Rating Management System. The system incorporates multiple layers of security mechanisms to safeguard sensitive information, maintain data integrity, and ensure operational continuity.

User Authentication

User authentication ensures that only authorized individuals can access the database system.

- Each user must log in using valid credentials (username and password).
- Authentication is verified before granting access to database resources.
- Invalid login attempts are rejected to prevent unauthorized access.

This mechanism ensures accountability and protects sensitive operations such as inserting, updating, or deleting reviews.

Role-Based Access Control (RBAC)

Role-Based Access Control is implemented to restrict database actions based on user roles. Different levels of access are defined, such as:

- **Administrator**
  - Full privileges (CREATE, DROP, ALTER, DELETE, GRANT, etc.)
  - Manages users and system configurations.
- **Moderator**
  - Can view and manage reviews.
  - Limited update and delete permissions.
- **Customer/User**
  - Can insert reviews and ratings.

- Can update or delete only their own reviews.
- Read-only access to product listings.

Privileges are assigned using MySQL's GRANT and REVOKE statements. This ensures users can perform only authorized actions, minimizing the risk of accidental or malicious data manipulation.

Data Integrity Protection

To maintain database reliability and prevent corruption:

- **Primary Key Constraints** ensure unique record identification.
- **Foreign Key Constraints** enforce valid relationships between tables.
- **NOT NULL and CHECK Constraints** prevent incomplete or invalid data entries.
- **UNIQUE Constraints** avoid duplication of sensitive fields.

These mechanisms collectively protect the accuracy and consistency of stored data.

Secure Password Storage

Passwords are stored securely using encryption or hashing techniques rather than plain text storage.

- Passwords are hashed before being stored in the database.
- Even database administrators cannot view original passwords.
- This protects user credentials in case of unauthorized database access.

Database Backup and Recovery

To ensure data safety and continuity, regular database backups are performed using the **mysqldump utility**.

Backup Process:

- The mysqldump tool exports the database structure and data into a .sql backup file.

- Backups can be scheduled periodically to minimize data loss risks.
- Backup files are stored securely to prevent unauthorized access.

Recovery Process:

- In case of system failure, hardware crash, or data corruption:
    - The backup file can be restored using MySQL restore commands.
    - The database is recreated with all tables, records, and relationships intact.
- This ensures minimal downtime and business continuity.

System Reliability and Continuity

By combining:

- Authentication and role-based access control
- Strong data integrity constraints
- Secure password storage
- Regular backup and restore mechanisms

the system ensures:

- Protection against unauthorized access
- Prevention of data corruption
- Recovery capability in case of failure
- Continuous availability of critical review and rating data

Overall, the security and backup strategies make the Product Review and Rating Management System robust, secure, and reliable for real-world implementation.

CHAPTER 12: FUTURE SCOPE AND CONCLUSION

Future Enhancements

- Integration with a web-based frontend
- Advanced role-based access control
- Advanced analytics and reporting
- Graphical dashboards for rating analysis

Conclusion

The Product Review and Rating Management System successfully demonstrates the application of MySQL for managing structured data. The project strengthens understanding of database design, normalization, transaction handling, and concurrency control, making it suitable for academic and real-world applications.

CHAPTER 13: REFERENCES

• MySQL Official Documentation

• DBMS Textbooks

• Online SQL Resources

CHAPTER 14: GLOSSARY

DBMS (Database Management System):

A software system used to store, manage, retrieve, and organize data efficiently in a structured format.

SQL (Structured Query Language):

A standard programming language used to interact with databases for creating tables, inserting data, retrieving records, and managing database operations.

Primary Key:

A field or set of fields that uniquely identifies each record in a table and ensures that no duplicate or null values are allowed.

Foreign Key:

A field in one table that refers to the primary key of another table, used to establish and maintain relationships between tables.

MySQL:

An open-source relational database management system widely used for building reliable, secure, and scalable database applications.